# Devoid Viewer

Codebase & Installer Documentation

This document explains every file, struct, method, and design decision in the Devoid Viewer codebase, and documents the Inno Setup installer. Written for someone new to Rust who wants to understand not only *what* the code does but *why* it was written that way.

## Table of Contents


| Section | Topic | Page |
|---|---|---|
| 1 | Project structure overview | 2 |
| 2 | Cargo.toml — dependencies explained | 2 |
| 3 | nav.rs — directory navigation | 4 |
| 4 | loader.rs — image decoding | 6 |
| 5 | main.rs — application core | 9 |
| 6 | Threading & asynchronous design | 20 |
| 7 | Key design decisions & trade-offs | 22 |
| 8 | Caveats & known limitations | 25 |
| 9 | Installer — devoid_viewer_setup.iss | 25 |
| 10 | Glossary of Rust concepts used | 28 |

# 1. Project Structure Overview

The project follows the standard Cargo layout:

| File | Responsibility |
|---|---|
| `Cargo.toml` | Project name, version, and all external dependencies. |
| `build.rs` | Build script — embeds devoid.ico and version metadata into the exe via winres. |
| `src/main.rs` | Event loop, app state, input handling, pixel renderer, window management. |
| `src/loader.rs` | Image decoding for all supported formats. Owns Frame and DecodedImage. |
| `src/nav.rs` | Directory scanning, human-sorted sibling list, next/prev navigation. |
| `devoid_viewer_setup.iss` | Inno Setup 6 installer script. |
| `devoid.ico` | Multi-size application icon (16, 32, 48, 256 px). Embedded in the exe. |

# 2. Cargo.toml — Dependencies Explained

Each crate was chosen for leanness. The dependency graph is roughly 50–70 crates and the release binary around 2–3 MB.

## Windowing: winit

**winit** handles the event loop, keyboard, window resize, drag-and-drop, and monitor queries. The rwh_05 feature enables the raw-window-handle v0.5 interface that softbuffer requires. On Windows the platform extension EventLoopBuilderExtWindows is used to install a Win32 message hook.

## Pixel display: softbuffer

**softbuffer** provides a Vec<u32> that maps directly to the window's pixels. No GPU pipeline, no shader compilation. On Windows it uses a GDI DIB section, compositor-accelerated without requiring a GPU context.

> ■ *DWM fills every new window with white before the application paints. Mitigated by hiding the window until the image is ready and painting gray synchronously before the event loop starts.*

## Native dialogs: windows-sys

**windows-sys** is a zero-overhead Windows API binding. Used for MessageBoxW (all dialogs), SystemParametersInfoW(SPI_GETWORKAREA) (precise taskbar exclusion), and PostMessageW (taskbar thumbnail close fix). Target-specific — does not affect non-Windows builds.

## Image decoding: image

The **image** crate handles BMP, PNG, JPEG, GIF, WebP, TIFF, and ICO. GIF animation uses image::codecs::gif::GifDecoder with AnimationDecoder. The ICO format is also used at startup to

decode devoid.ico for the window icon.

## JPEG XL: jxl-oxide

Pure-Rust JPEG XL decoder. Not yet part of the image crate. Uses the low-level streaming API.

## SVG: resvg + usvg + tiny-skia

- **usvg** parses and normalises the SVG scene tree.
- **resvg** renders into a tiny-skia Pixmap.
- **tiny-skia** rasterises to premultiplied RGBA, which is then unpremultiplied to straight RGBA.

## Channel communication: crossbeam-channel

Faster and more ergonomic than std::sync::mpsc. Bounded at capacity 32 to throttle background threads if the UI falls behind.

## Windows resource embedding: winres (build dependency)

Runs at build time only. Embeds devoid.ico and version metadata into the exe. Zero runtime cost.

# 3. nav.rs — Directory Navigation

Maintains a human-sorted list of sibling image files in the same directory as the currently open file. Navigation wraps at both ends.

### struct DirNavigator

- **files: Vec<PathBuf>** — human-sorted sibling list.
- **index: usize** — position of the currently displayed file.

Derives **Default** — auto-generated empty initialiser.

### fn set_current(path)

Called on every image open. Compares the new path's parent directory against the first entry in the current list to decide whether to rebuild. Falls back to a single-entry list if the path is not found — this handles files opened from archiver temp folders that contain only one extracted file.

### fn next() / fn prev()

Both return None when fewer than 2 images exist — navigation on a single-image directory is a silent no-op. Both wrap around using modulo and checked_sub for safe unsigned arithmetic.

### fn peek_next() / fn peek_prev()

Read-only lookahead used by preload_adjacent(). Returns None for single-image directories so no unnecessary decode threads are spawned.

### fn remove(path)

Removes a path after deletion. Uses saturating_sub to keep the index in bounds when the deleted file was the last one.

### fn rename(old_path, new_path)

Replaces a path in the sibling list after a successful rename. Preserves the current index so navigation order is unaffected. Called from App::apply_rename() after the user confirms a rename dialog.

### fn current()

Returns the currently active path, which may differ from the last set_current argument if remove was called.

### fn human_sort / fn consume_number

Splits each filename into alternating text and digit runs. Digit runs are parsed as u64 and compared numerically, so *img2.png* sorts before *img10.png*. saturating_mul and saturating_add prevent overflow on pathologically long digit sequences.

# 4. loader.rs — Image Decoding

Owns the image type definitions and all decoding logic. Returns **DecodedImage** directly — no conversion needed in main.rs. No knowledge of winit, softbuffer, or UI state.

## struct Frame

**pixels: Vec<u8>** — straight RGBA8, row-major. **delay_ms: u32** — frame duration. Zero for non-animated formats.

## struct DecodedImage

One or more frames sharing the same canvas dimensions. **is_animated()** returns true when more than one frame exists. **frame(idx)** borrows a specific frame. **single()** is a private constructor for single-frame images.

## enum LoadOutcome

- **Ok(DecodedImage)** — decoded successfully using the declared extension.
- **WrongExtension(DecodedImage, String)** — decoded via header sniffing; the String is the correct extension (e.g. 'jpg').
- **Unsupported(String)** — not a recognised image format even after header sniffing; the String is a human-readable reason.

## fn load_image(path) → LoadOutcome

Two-pass strategy: first tries the declared extension (fast path); on failure calls sniff_format() to read the file header. If a different format is detected, decodes with DecodeStrategy::ByContent and returns WrongExtension. If the same extension is detected but decoding still fails, the file is corrupt. If no format is detected at all, returns Unsupported.

## enum DecodeStrategy

Passed to load_raster() to select how the image format is determined. **ByExtension** calls image::open() which trusts the file extension — the fast, normal path. **ByContent** opens the file and calls with_guessed_format() which reads magic bytes from the header, completely ignoring the extension. Used on the fallback path when the extension was found to be wrong.

> ■ *An enum is used rather than a bool parameter because load_raster(path, DecodeStrategy::ByContent) is self-documenting at the call site, whereas load_raster(path, true) tells the reader nothing.*

## fn load_by_ext(ext, path, strategy)

Dispatches to the correct decoder: svg → load_svg, jxl → load_jxl, gif → load_gif, everything else → load_raster. The strategy parameter is only meaningful for the raster path.

## fn sniff_format(path)

Reads the first 32 bytes of the file — once — and passes those bytes to both the image crate's format guesser and manual signature checks for JXL and SVG. 32 bytes is sufficient for all magic numbers used by the supported formats. Returns a lowercase extension string or None.

- JXL codestream: starts with FF 0A.
- JXL ISO BMFF container: starts with 00 00 00 0C 4A 58 4C 20 0D 0A 87 0A.
- SVG: first non-whitespace byte after optional UTF-8 BOM is '<'.

## fn load_raster — BMP, PNG, JPEG, WebP, TIFF, ICO

The ByExtension path calls image::open(path), which selects the decoder by file extension. The ByContent path wraps the file in a BufReader and calls with_guessed_format() so the decoder is selected by the file header regardless of extension.

## fn load_gif — Animated GIF

Uses GifDecoder with AnimationDecoder. Per-frame delay comes from frame.delay().numer_denom_ms() with a 20 ms minimum to prevent zero-delay GIFs from locking up the event loop, and a 100 ms fallback when the denominator is zero.

## fn load_jxl — JPEG XL

- Requests sRGB output regardless of embedded colour space.
- Renders frame 0 and streams f32 samples.
- Converts to RGBA8 via f32_to_u8 (clamp then scale). Alpha is pre-filled to 255 so 1-channel and 3-channel files only need to write RGB.

## fn load_svg — SVG

- usvg parses and normalises the scene tree.
- resvg renders to a tiny-skia Pixmap (premultiplied RGBA).
- Each pixel is unpremultiplied by dividing R, G, B by alpha. Fully transparent pixels are zeroed.

# 5. main.rs — Application Core

Contains the winit event loop, all application state, and the pixel renderer. No widget toolkit, no font renderer, no GPU pipeline.

## 5.1 Constants

- **BG: u8 = 42** — background grey used in the blank state, alpha compositing, and the pre-loop synchronous surface fill.
- **APP_NAME** — application name shown in the title bar when no image is open; used in all dialog captions so it is defined once.
- **DOUBLE_CLICK: Duration** — maximum gap between two left-clicks that counts as a double-click.

## 5.2 enum Msg

Five variants posted from background threads to the UI thread:

- **Loaded** — primary decode finished. Carries path, image, and OpenMode.
- **WrongExtension** — decoded via header sniff. Also carries the correct extension string so the UI can offer a rename dialog.
- **Preloaded** — speculative neighbour decode for the preload cache.
- **Unsupported** — file is not a recognised image format.
- **Deleted** — async deletion result: Ok(()) or Err(String).

## 5.3 struct WakeUp / type Proxy

**WakeUp** is a zero-size marker type injected by background threads to wake the sleeping event loop after posting a Msg. **Proxy** is a type alias for the verbose EventLoopProxy<WakeUp>.

## 5.4 struct DrainResult

Everything the event loop needs to know after one drain of the message channel. Named fields are self-documenting at call-sites:

- **resize_to: Option<(u32, u32)>** — set when the first image loaded; the window should be resized to these dimensions.
- **error_msg: Option<String>** — set when any error occurred (unsupported file or failed deletion). The window must be revealed before this dialog is shown.
- **rename_offer: Option<(PathBuf, String)>** — set when a file decoded under the wrong extension; the rename dialog is shown after the drain loop so it never blocks inside the loop.

## 5.5 enum OpenMode

**FirstImage** — the window resizes to fit. **Navigate** — the window stays the same size. Carried through Msg::Loaded so the resize intent is explicit at the point where the message is constructed.

## 5.6 struct App — fields

- **current_path / current_image** — the displayed file. current_image is not cleared when navigation starts; the previous image stays on screen until the new one is ready.
- **frame_index / next_frame_at** — GIF animation state.
- **navigator** — the DirNavigator for the current directory.
- **msg_tx / msg_rx** — the crossbeam channel endpoints.
- **loading_path** — the path currently being decoded. Used to discard stale Loaded messages if the user navigated away before decoding finished.
- **preload_cache** — decoded images for neighbouring files.
- **deleting** — prevents a second Delete keypress while a deletion is in flight.
- **pending_center** — set when a resize is issued; cleared on the next WakeUp when the resize has landed and center_window() can be called accurately.
- **pre_maximize** — physical inner size saved before entering managed-maximise. Position is not saved — we always centre on restore.
- **maximized / fullscreen** — independent boolean flags for the two window modes.
- **last_click** — timestamp of the last left-click for double-click detection.

## 5.7 fn current_filename()

Returns the filename component of current_path as an owned String, for the title bar and delete dialog. Defined once on App, used in both places.

## 5.8 fn register_click()

Records the current timestamp and returns true when the interval since the last click is within DOUBLE_CLICK. Used by the MouseInput handler.

## 5.9 fn toggle_maximize()

Saves the physical inner size before calling window.set_maximized(true). On restore, calls window.request_inner_size(size) with the saved physical size — since request_inner_size is asynchronous, sets pending_center = true so centering fires after the resize lands. A no-op when fullscreen is true.

## 5.10 fn toggle_fullscreen() / fn exit_fullscreen()

toggle_fullscreen() calls window.set_fullscreen(Some(Fullscreen::Borderless(None))) or None. Borderless uses the current monitor. Independent of managed-maximise. exit_fullscreen() exits only if currently fullscreen, returns true if a change was made.

## 5.11 fn open_image()

Cache hit: calls apply_image directly. Cache miss: seeds the navigator immediately so peek_next/peek_prev are accurate for preloading, sets loading_path, then spawns a background thread.

### 5.12 fn apply_image() → Option<(u32, u32)>

Stores the image and path, clears loading_path, updates the navigator, starts preloading adjacent files, and resets animation. Returns image dimensions only when OpenMode::FirstImage.

### 5.13 fn preload_adjacent()

Spawns up to two background decode threads for the next and previous siblings. Only caches LoadOutcome::Ok results — wrong-extension files would prompt a rename dialog and must not be silently preloaded. Does nothing for single-image directories.

### 5.14 fn navigate()

Calls navigator.next() or navigator.prev(), both of which return None for single-image directories. Triggered by arrow keys, Page Up/Down, and the mouse wheel.

### 5.15 fn request_delete() / fn on_deleted()

request_delete() spawns a thread that calls std::fs::remove_file. The result arrives as Msg::Deleted. on_deleted() removes the path from the navigator, clears the preload cache, and navigates to the next sibling or clears the display. Returns Option<String> — the error message if deletion failed, None on success. The caller assigns this to DrainResult.error_msg, keeping on_deleted decoupled from DrainResult.

### 5.16 fn drain_messages() → DrainResult

Non-blocking try_recv loop. All blocking side-effects (dialogs, renames) are returned in DrainResult rather than performed inside the loop. Loaded messages whose path does not match loading_path are silently discarded — this handles the race where the user navigates faster than images decode.

### 5.17 fn apply_rename()

Updates current_path and calls navigator.rename() after a successful rename. Called from the event loop after offer_rename() returns the new path.

### 5.18 Rendering functions

- **fit_scale()** — largest uniform scale that fits the image without upscaling.
- **nearest()** — point-samples an RGBA8 buffer at float coordinates.
- **composite()** — alpha-blends a straight-RGBA pixel over BG.
- **pack_rgb()** — packs three u8 values into softbuffer's 0x00RRGGBB u32.
- **render()** — double for-loop (row then column). dx/dy are cheap loop counters, not computed from a flat index. Always paints unconditionally so off-screen regions are correct when the window is moved back on-screen.

### 5.19 Native dialogs

- **to_wide()** — encodes a &str as null-terminated UTF-16 for Win32 API calls. Defined once, used by all four dialog functions.
- **confirm_delete_dialog()** — MB_YESNO warning.
- **show_error_dialog()** — MB_OK error. Caption is APP_NAME.
- **offer_rename()** — checks parent directory read-only status, calls unique_rename_target() to avoid clobbering existing files, shows MB_YESNO question. Returns Some(new_path) on success.
- **unique_rename_target()** — finds the first non-existing path of the form 'stem.ext', 'stem (1).ext', 'stem (2).ext', etc. Matches Windows Explorer's own naming convention for duplicates.

## 5.20 Window helpers

- **work_area()** — usable desktop area. Windows: SPI_GETWORKAREA. Other platforms: 90 % × 85 % of monitor.
- **clamp_to_work_area()** — scales dimensions down to fit; never up.
- **center_window()** — centres on current monitor with 30 px minimum top margin.
- **update_title()** — sets 'filename — Devoid Viewer' or just 'Devoid Viewer' when no image is open.
- **scroll_is_forward()** — interprets LineDelta and PixelDelta as a navigation direction. Down or right = next, up or left = previous.

## 5.21 fn main() — startup sequence

- **Raw argument path** — never canonicalized. std::fs::canonicalize prepends \\?\ on Windows, which breaks the image crate and causes silent failures for archiver temp paths.
- **Header probe** — image_size_hint() reads just the file header for dimensions so the window appears at the correct size before the full decode finishes.
- **SC_CLOSE hook** — taskbar thumbnail × button sends WM_SYSCOMMAND/SC_CLOSE which winit 0.29 does not translate into CloseRequested. Re-posted as WM_CLOSE.
- **SC_MAXIMIZE hook** — suppressed so toggle_maximize() is the single code path for maximise, giving unified behaviour with the client-area double-click.
- **start_hidden** — window is hidden when an image argument is provided; revealed after resize and centering are applied via the two-WakeUp sequence.
- **Self-WakeUp after resize** — after setting pending_center = true, the event loop sends itself a WakeUp to guarantee the reveal sequence completes even when no preload threads fire — as happens for a single image in an archiver temp folder.
- **process::exit(0) on close** — background threads hold channel senders and would delay a graceful exit. Immediate termination is correct for a stateless viewer.
- **Mouse wheel navigation** — TouchPhase::Cancelled is filtered; all other phases are treated as active scrolling because winit reports standard mouse-wheel events as TouchPhase::Moved.

# 6. Threading & Asynchronous Design

## 6.1 The channel pipeline

- One bounded crossbeam channel, capacity 32.
- msg_tx is cloned cheaply for each background thread.
- msg_rx is drained non-blockingly on every WakeUp.
- Background threads post a Msg then call proxy.send_event(WakeUp).

■ *proxy.send_event() is thread-safe — EventLoopProxy is Arc-based.*

## 6.2 ControlFlow — zero idle CPU

ControlFlow::Wait sleeps until any event arrives. ControlFlow::WaitUntil wakes at the next GIF frame deadline. When displaying a still image the process uses effectively zero CPU.

## 6.3 Header probe + async decode

image_size_hint() reads only the file header. The window is shown at the correct size with a gray background. The full decode runs on a background thread. Startup is fast regardless of file size.

## 6.4 The two-WakeUp window reveal sequence

Window hidden → gray paint → decode thread starts → WakeUp arrives → resize issued → pending_center = true → self-WakeUp sent → next WakeUp: center_window() → set_visible(true) → RedrawRequested paints the image. The self-WakeUp is essential when opening from an archiver temp folder containing only one file, because no preload threads fire to deliver the second WakeUp.

## 6.5 Stale result discarding

Loaded and WrongExtension messages are checked against loading_path. A mismatch means the user navigated away before the decode finished — the message is silently dropped.

## 6.6 Drain loop is never blocking

All side-effects that could block (dialogs, renames, error display) are collected in DrainResult and performed after the drain loop returns. on_deleted() returns Option<String> rather than showing a dialog inline for the same reason.

## 6.7 Extension-mismatch recovery

load_image() returns LoadOutcome::WrongExtension when a file decodes successfully under a different format than its extension suggests. The image is displayed immediately. The rename offer dialog appears after the drain loop so it never delays showing the image. If the rename succeeds, apply_rename() updates current_path and the navigator so subsequent operations (deletion, navigation) use the correct path, and update_title() reflects the new filename.

## 6.8 Rename collision avoidance

unique_rename_target() checks whether the target path already exists before proposing it. If '5.jpg' already exists when renaming '5.png', the proposed name becomes '5 (1).jpg', matching Windows Explorer's convention. The counter increments until a free name is found.

## 6.9 Immediate process exit

Background threads hold msg_tx clones. A graceful exit would wait for all threads to finish — potentially seconds for a large image. std::process::exit(0) terminates immediately. For a stateless viewer with nothing to save, this is always correct.

## 6.10 The canonicalize trap

std::fs::canonicalize prepends \\?\ on Windows. The image crate does not handle this prefix reliably on all code paths, causing silent decode failures. Archivers like 7-Zip pass the temp path directly — canonicalizing it adds no value and causes harm. The raw argument is used throughout.

■ *Never pass a canonicalized path to the image crate on Windows.*

# 7. Key Design Decisions & Trade-offs

### winit + softbuffer over a GUI toolkit

A full toolkit adds ~200+ crates and ~9 MB. winit + softbuffer totals ~50–70 crates and ~2–3 MB. The trade-off is that native OS dialogs must be called directly via windows-sys.

### Fit-to-window rendering

Every image is scaled to fit the current window using fit_scale(), always maintaining aspect ratio. The window is the zoom control. No hidden zoom state, no scroll position — every image is always fully visible.

### Nearest-neighbour sampling

Point sampling is correct and fast at fit-to-window scale where no magnification occurs. Bilinear filtering would add complexity with no visible benefit.

### DecodeStrategy enum over a bool parameter

load_by_ext(&ext, path, DecodeStrategy::ByContent) is self-documenting at the call site. load_by_ext(&ext, path, true) tells the reader nothing without looking up the function signature.

### on_deleted returns Option<String>

Previously on_deleted took &mut DrainResult as a parameter, coupling it to a type it had no business knowing about. Returning the error message as Option<String> keeps each function's responsibility clear.

### Single-image navigation is a no-op

navigator.next() and navigator.prev() return None when fewer than 2 images exist. This applies to keyboard, mouse wheel, and trackpad navigation. Prevents a redundant reload of the same file.

### No dirty flag — always paint on RedrawRequested

Always painting is correct: when the window is partially off-screen and then moved back, the next redraw writes fresh pixels for the entire window. With ControlFlow::Wait the event loop only delivers RedrawRequested when something has actually changed.

### update_title called on every WakeUp

Rather than tracking whether the title needs updating, update_title() is called unconditionally on every WakeUp. window.set_title() is a no-op when the title hasn't changed, so this is correct and costs nothing.

### Title bar shows filename on rename

After offer_rename() succeeds, apply_rename() updates the navigator and current_path, then update_title() is called again. The user sees the new filename immediately without any additional state tracking.

## TouchPhase::Cancelled only

The MouseWheel handler filters only TouchPhase::Cancelled rather than allowing only Started/Moved/Ended. This is because winit reports standard mouse-wheel events as TouchPhase::Moved regardless of device type — filtering the active phases would incorrectly block physical wheels.

## SC_MAXIMIZE suppression

The title-bar double-click sends SC_MAXIMIZE to the Win32 message pump. Suppressing it in the msg_hook and relying on winit's MouseInput event (which fires for the same gesture) gives unified maximise behaviour: both the client-area double-click and the title-bar double-click call toggle_maximize(), which enforces the fullscreen guard and saves the pre-maximize size.

# 8. Caveats & Known Limitations

### No status bar

Filename, image index, and dimensions are not displayed in the window. Showing text requires a font rasteriser crate. The filename is shown in the title bar instead.

### Animated GIF only

Animation is supported for GIF. APNG and animated WebP or AVIF would need separate decoder paths.

### Render loop is O(win_w × win_h)

The renderer visits every window pixel on every redraw using simple loop-counter arithmetic — no division or modulo per pixel.

### JXL HDR clamped, not tone-mapped

JPEG XL supports HDR content. f32_to_u8 clamps values above 1.0 to 255, losing highlight detail. Standard JXL photos are unaffected.

### SVG external resources not loaded

usvg with default options does not resolve external fonts or linked images. SVGs that depend on external resources render with missing content.

### Work area heuristic on non-Windows

The precise SPI_GETWORKAREA path is Windows-only. Other platforms use 90 % × 85 % of the monitor size, which may not account for taskbars or docks accurately.

### Navigation from archiver temp folders

When an image is opened from an archiver the temp folder typically contains only the one extracted file. Arrow keys, Page Up/Down, and mouse wheel are silent no-ops. This is correct behaviour.

### SC_MAXIMIZE winit behaviour dependency

The title-bar double-click is unified with the client-area double-click by relying on winit firing MouseInput for the title-bar gesture. This is observed behaviour rather than a documented guarantee and could change in a future winit version.

# 9. Installer — devoid_viewer_setup.iss

Built with Inno Setup 6. Place the script in the project root alongside Cargo.toml, build the release binary first, then run iscc devoid_viewer_setup.iss. Output: Output\DevoidViewer_Setup.exe.

## 9.1 Why Inno Setup instead of WiX 3

WiX 3 evaluates Component Condition elements during the MSI costing phase, before any UI is shown. This makes it impossible to conditionally write to HKLM or HKCU based on the user's scope choice — the condition fires before the scope dialog appears. Inno Setup handles this correctly via IsAdminInstallMode and the HKA registry root.

## 9.2 MSI vs EXE installer

Inno Setup produces an EXE installer. MSI stores uninstall information centrally in Windows Installer's database so the application folder stays clean. EXE installers place unins000.exe and unins000.dat in the installation folder. This is standard — Notepad++, 7-Zip, and many others work the same way.

## 9.3 Installation scope

PrivilegesRequired=lowest with PrivilegesRequiredOverridesAllowed=dialog gives the user a choice: Install just for me (no UAC) or Install for everyone (UAC prompt). The HKA registry root resolves to HKLM for per-machine and HKCU for per-user — one script, two scopes.

## 9.4 App Paths registration

Registers Software\Microsoft\Windows\CurrentVersion\App Paths\DevoidViewer.exe in HKLM or HKCU. Lets Windows and archivers like 7-Zip locate and launch the application via ShellExecuteEx from any context.

## 9.5 File associations

| Feature | Registry written | Notes |
|---|---|---|
| ProgId | HKA\Software\Classes\DevoidViewer.ImageFile | Always installed. Defines launch command and icon. |
| Open With | HKA\...\OpenWithProgIds | Adds to Open With menu without changing the current de |
| Set as Default | HKLM\Software\Classes\.ext | Only on per-machine installs (Check: IsAdminInstallMode) |

## 9.6 The UserChoice problem

Windows writes a UserChoice entry when HKCU\Software\Classes\.ext is set. UserChoice contains a cryptographic hash that Windows verifies on every file open. Writing the default association only to HKLM avoids generating UserChoice entries entirely, so uninstalling leaves no stale associations pointing at a deleted ProgId.

## 9.7 Uninstaller behaviour

- taskkill closes any running Devoid Viewer instances before files are deleted, preventing the exe from being locked.
- The install folder is always removed via uninsalwaysuninstall.
- All UserChoice keys for the 11 supported extensions are deleted on uninstall.
- Shell icon cache refreshed via ie4uinit.exe on both install and uninstall.

## 9.8 Supported extensions

jpg, jpeg, png, bmp, gif, webp, tiff, tif, ico, jxl, svg.

# 10. Glossary of Rust Concepts Used

**Ownership**
Every value has exactly one owner. When the owner goes out of scope the value is dropped. No garbage collector needed.

**Borrowing**
Lend a reference (&T;) without transferring ownership. References cannot outlive the value they point to.

**&mut; self vs &self;**
&mut; self gives exclusive mutable access. &self; gives shared read-only access. The compiler enforces that both cannot coexist.

**Option**
Either Some(value) or None. The safe replacement for null pointers.

**Result**
Either Ok(value) or Err(error). The ? operator propagates errors automatically up the call stack.

**Vec**
A growable heap-allocated array. Used for pixel buffers, frame lists, and path lists.

**HashMap**
A hash table. O(1) average lookup. Used for the preload cache keyed by PathBuf.

**Arc**
Atomically Reference Counted smart pointer. Allows shared ownership across threads. Used for the Window handle.

**Enum with data**
Rust enums carry data per variant. Msg::Loaded carries a PathBuf, a DecodedImage, and an OpenMode.

**then_some**
(condition).then_some(value) returns Some(value) if true, None otherwise.

**Lifetime**
A compile-time label on a reference. View<'a> borrows from App — the compiler ensures View cannot outlive App.

**Closure**
An anonymous function. The move keyword transfers ownership of captured variables into the closure.

**match**
Exhaustive pattern matching. Every variant of an enum must be handled or the code will not compile.

**#[derive(...)]**
Auto-generates trait implementations. #[derive(Clone, Debug)] writes clone() and the debug formatter for free.

**Send**
Marker trait meaning a type is safe to transfer to another thread.

**ControlFlow**
Winit enum controlling the event loop sleep behaviour. Wait sleeps until any event. WaitUntil sleeps until a deadline.

**EventLoopProxy**
Thread-safe handle to the event loop. proxy.send_event() injects a synthetic event from any thread.

**NonZeroU32**
A u32 guaranteed to be non-zero at compile time. softbuffer's resize method requires this.

**cfg(windows)**
Conditional compilation attribute. Code inside #[cfg(windows)] is compiled only on Windows.

**unsafe**
Opts into calling C API functions. Used for MessageBoxW, SystemParametersInfoW, and PostMessageW.

**type alias**
type Proxy = ...; creates a shorter name with no runtime cost.

**Zero-size struct**
struct WakeUp; occupies zero bytes. Used as a type-level marker to wake the event loop.

**include_bytes!**
Embeds a file's raw bytes into the binary at compile time. Used for devoid.ico.

**process::exit(0)**
Terminates the process immediately, bypassing Rust's normal drop sequence. Used on window close to avoid background threads delaying exit.

**DecodeStrategy enum**
Replaces a bool parameter in load_by_ext. ByExtension and ByContent are self-documenting at every call site.

**msg_hook**

winit 0.29 Windows platform extension. Intercepts every Win32 message before winit sees it. Used for SC_CLOSE and SC_MAXIMIZE.

**unique_rename_target**

Finds the first non-existing path of the form 'stem.ext', 'stem (1).ext', etc. Prevents overwriting an existing file on rename.